

AeroStream Capstone – Batch Processing & MLOps Pipeline

Introduction

The goal of the first phase of this capstone project was to analyze a massive public transportation dataset from the US Bureau of Transportation Statistics. While the original approach relied on legacy Hadoop and Hive infrastructure, this project has been entirely modernized into a production-ready Machine Learning Operations (MLOps) pipeline.

This report outlines the data engineering pipeline used to extract, clean, and analyze the data, the transition from heavy map-reduce queries to PySpark analytics, and the implementation of an automated CI/CD pipeline.

System Overview & Architecture

The original architecture relied on AWS EMR, Hive, and DynamoDB. To modernize the pipeline, reduce cloud vendor lock-in, and improve processing speeds, the architecture was refactored:

- **Data Ingestion & Cleaning:** Raw data is read from AWS S3, cleaned using robust Python pipelines, and converted from uncompressed CSVs into highly optimized **Parquet** files. This reduced storage footprint and vastly improved read speeds.
- **Infrastructure as Code (IaC):** The cloud infrastructure is defined using the Serverless Framework (`serverless.yml`), replacing overly broad IAM roles with strict, least-privilege security configurations for AWS Lambda and S3.
- **Analytics Engine:** Heavy Hive SQL queries were replaced with **PySpark DataFrames**. This eliminated the need to manually configure Hadoop MapReduce tasks or tune cluster memory allocations, leaning on Spark's automated query optimizer.
- **Continuous Integration (CI/CD):** A GitHub Actions workflow (`ci.yml`) was introduced to automatically test the pipeline using `pytest`, ensuring code quality via `black` and `flake8`.

linting on every push.

Optimizations

1. **Parquet Conversion:** By migrating the storage layer from raw text (CSV) to a columnar format (Parquet), subsequent analytical queries skipped parsing overhead and read only the necessary columns.
2. **PySpark over Hive:** Shifting from Hive to PySpark eliminated the heavy disk I/O of traditional MapReduce, utilizing Spark's in-memory processing.
3. **Local Containerization:** Rather than spending money on an active AWS EMR cluster during development, the entire environment was containerized using Docker, allowing for rapid local iteration.

Results: Exploratory Data Analysis

(Note: The underlying data remains the same, so the analytical findings hold true).

Group 1: Top Busiest Airports & Carrier Delays

- **ORD (Chicago O'Hare)** ranked as the busiest airport with over 12.4 million flights, closely followed by ATL (Atlanta).
- **Hawaiian Airlines (HA)** emerged as the most punctual airline, averaging a negative delay (-1.01 minutes), meaning flights frequently arrived early.
- Saturday proved to be the best day to fly, averaging only 4.3 minutes of delay, whereas Fridays were the worst at 9.72 minutes.

Group 2: Route & Destination Analytics

- Analysis of specific routes revealed stark contrasts in performance. For example, flights from CMI to ORD averaged over 10 minutes of delay, while flights from LAX to LAX (reroutes/test flights) averaged early arrivals.

Conclusion & MLOps Future

The financial impact of delayed flights—ranging from missed arrival slots to severe reputation damage—makes this data incredibly valuable. Travel agencies can leverage these exact PySpark aggregations to recommend better routes to customers.

Moving forward, this batch pipeline acts as the foundational feature-engineering layer for the next phase of the project: training an XGBoost predictive model tracked by MLflow to actively predict flight delays before they occur.